

A New Adaptive Maneuvering Target Tracking Algorithm Using Artificial Neural Networks

Zhijun Yu, Jianming Wei, and Haitao Liu

Abstract—A new neural network (NN) aided adaptive unscented Kalman filter (UKF) is presented for tracking high maneuvering target. In practice, the dynamic systems of many target tracking problems are usually nonlinear and incompletely observed, moreover, there may be large modeling errors when the target is maneuverable or some parameters of the system models are inaccurate or incorrect. The adaptive capability of filters is known to be increased by incorporating a neural network into the filtering procedure. On the other hand, some nonlinear filtering methods such as extended Kalman filter (EKF) have been used to train a NN with fast convergence speed by augmenting the state with unknown connecting weights. Tackling the natural coalescent between the filtering algorithm and the NN described above, first a more efficient learning algorithm based on unscented Kalman filter (UKF) is derived, which can give a more accurate estimate of the weights and possess faster convergence rate. We then extend the algorithm to form a new NN aided adaptive UKF algorithm and use it in maneuvering target tracking applications. The NN in this algorithm is used to approximate the uncertainty of system models and is trained online, together with the target state estimation. Some simulations are also given to validate that the proposed method can give well state estimation of a highly maneuvering target.

I. INTRODUCTION

In the past few decades, the problem of state estimation in maneuvering target tracking has attracted attention in several fields such as air defence, radar tracking, robot vision and ground sensor network applications [1]-[3]. For many target tracking problems, the state model or the measurement model is nonlinear, the target tracking problem becomes a nonlinear filtering and estimating problem and analytical solution is unable to obtain. There have been many suboptimal filtering methods used for non-linear state estimation problem. The classical method is extending the standard Kalman filter (KF) to nonlinear system by local nonlinearizing all nonlinear models around certain points, which is so called extended Kalman filter (EKF) [4], [5]. Unfortunately, the EKF has two fatal drawbacks: First, the derivation of the Jacobian matrices, the linear approximators to the nonlinear functions, can be complex causing implementation difficult. Second, these linearizations can lead to filter instability when the models are

highly nonlinear and the local linearity assumption breaks down [6]. To address these limitations, Julier and Uhlmann proposed the unscented Kalman filter (UKF) [7]. The UKF operates on the premise that it is easier to approximate a Gaussian distribution than it is to approximate an arbitrary nonlinear function. Instead of linearizing using Jacobian matrices, the UKF using a deterministic sampling approach to capture the mean and covariance estimates with a minimal set of sample points. Thanks to better convergence properties and simplicity of design, the UKF is preferred over the traditional EKF. It can capture the posterior mean and covariance of the hidden state accurately to the second order for any nonlinearity, with errors only introduced in the third orders and more [8]-[10].

In most practical target tracking applications, the target dynamic model and some important parameters are hard to get accurately in advance, and the target also may be maneuverable. The highly uncertainty and incompleteness of information will weaken UKF's advantage of handling any functional nonlinearity. It is known that the adaptive capability of the KF (also EKF and UKF) can be increased by incorporating a neural network into the standard KF [11], [12]. Neural network can approximate any nonlinear system to a desired degree of accuracy, or predict a non-stationary process if the weights are adaptively adjusted during online update and has been widely used in nonlinear dynamic system modelling. A conventional learning algorithm for forward neural network is backward propagation (BP) algorithm, which is a first-order steepest decent method. It iteratively adjusts the connecting weights to minimize the differences between the outputs of the NN and the desired outputs. However, the convergence speed is slow and may not be effective. In [13] and [14], the EKF has been used to train the NN by treating the weights of a network as the state of the nonlinear dynamic system.

For the advantages of UKF algorithm described above, we will first propose a UKF-based algorithm for NN training, and then extend the algorithm further to develop an NN-aided adaptive UKF algorithm for maneuvering target tracking applications.

The rest of the paper is organized as follows. In the next section we explain the basic process of the UKF algorithm used for NN training. In Section III, we introduce the proposed NN aided adaptive UKF for maneuvering target tracking. Section IV presents simulations of comparing the proposed algorithm with interactive multiple model algorithms (IMM) [2] for tracking a maneuvering target in

Zhijun Yu is with the Shanghai Institute of Microsystem and Information Technology, Chinese Academy of Science, Shanghai, CO 200050 China (corresponding author, e-mail: seawave.yu@ yahoo.com).

Jianming Wei is with the Shanghai Institute of Microsystem and Information Technology, Chinese Academy of Science, Shanghai, CO 200050 China.

Haitao Liu is with the Shanghai Institute of Microsystem and Information Technology, Chinese Academy of Science, Shanghai, CO 200050 China.

sensor network field. Finally, a conclusion of the work is given in Section V.

II. UKF ALGORITHM FOR NEURAL NETWORK TRAINING

UKF was first introduced by Julier and Uhlmann. The main advantage of UKF is that it does not use any linearization for calculating the state prediction and covariance, which leads to more accurate Kalman gain estimation and better state estimates.

A typical multilayer neural network with a single hidden layer is shown in Fig. 1. The L inputs x^i ($i=1,2,\dots,L$) and N outputs y^j ($j=1,2,\dots,N$) of the network are related through the connecting weights and the mapping function of $g(\bullet)$. To apply UKF to NN training, the first step is to organize all the inputs, outputs and network weights as state vectors. The UKF estimates the weights of the NN, which in turn are used to modify the state estimate predictions of the filter as a state estimation problem with the following dynamic and observation equations:

$$w_k = w_{k-1} \quad (1)$$

$$d_k = y_k + v_k = g(w_k, x_k) + v_k \quad (2)$$

where $x = (x^1 \ x^2 \ \dots \ x^L)^T$, $y = (y^1 \ y^2 \ \dots \ y^N)^T$, and $w = [(w^1)^T \ (w^2)^T]^T$. w is the vector containing all the link weights of the neural network as shown in Fig. 1. d and y are the desired and the true outputs, respectively. v is the random observation noise which is usually modeled as zero-mean Gaussian and white with the covariance matrix R . For this three-layer neural network, the n^{th} output of the NN in Fig. 1 can be written as

$$g_n(w_k, x_k) = \sum_{m=1}^M w_{mn}^2 \left(\sum_{l=1}^L w_{lm}^1 x^l \right), \quad n=1, 2, \dots, N \quad (3)$$

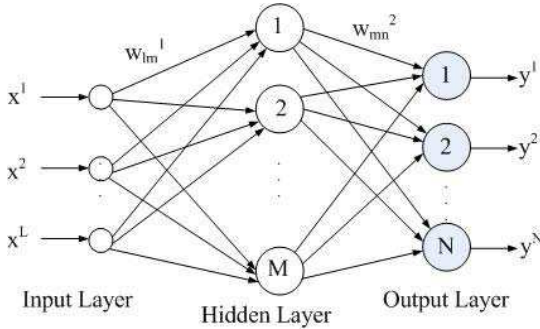


Fig. 1. The structure of a typical three-layer neural network.

The generic UKF algorithm for NN training can be summarized as follows:

a. Initialization,

$$\bar{w}_0 = E[w_0] \quad P_0 = E[(w_0 - \bar{w}_0)(w_0 - \bar{w}_0)^T] \quad (4)$$

b. Calculate sigma points,

$$\begin{cases} \chi_{k|k}^{(0)} = \bar{w} \\ \chi_{k|k}^{(i)} = \bar{w} + (\sqrt{(n+\lambda)P_{k|k}})_i, i=1,\dots,n \\ \chi_{k|k}^{(i)} = \bar{w} - (\sqrt{(n+\lambda)P_{k|k}})_i, i=n+1,\dots,2n \end{cases} \quad (5)$$

$$\begin{cases} W_0^{(m)} = \lambda / (n + \lambda) \\ W_0^{(c)} = \lambda / (n + \lambda) + (1 - \alpha^2 + \beta) \\ W_i^{(m)} = W_i^{(c)} = 0.5 / (n + \lambda), i=1,2,\dots,2n \end{cases} \quad (6)$$

c. Time update,

$$\chi_{k+1|k}^{(i)} = \chi_{k|k}^{(i)}, \quad \bar{w}_{k+1|k} = \sum_{i=0}^{2n} W_i^{(m)} \chi_{k+1|k}^{(i)} \quad (7)$$

$$P_{k+1|k} = \sum_{i=0}^{2n} W_i^{(c)} (\chi_{k+1|k}^{(i)} - \bar{w}_{k+1|k})(\chi_{k+1|k}^{(i)} - \bar{w}_{k+1|k})^T \quad (8)$$

$$\mathcal{J}_{k+1|k}^{(i)} = g(\chi_{k+1|k}^{(i)}, v_k), \quad \bar{y}_{k+1|k} = \sum_{i=0}^{2n} W_i^{(m)} \mathcal{J}_{k+1|k}^{(i)} \quad (9)$$

d. Measurement update,

$$P_{x_{k+1} y_{k+1}} = \sum_{i=0}^{2n} W_i^{(c)} (\mathcal{J}_{k+1|k}^{(i)} - \bar{y}_{k+1|k})(\mathcal{J}_{k+1|k}^{(i)} - \bar{y}_{k+1|k})^T + R_k \quad (10)$$

$$P_{x_{k+1} y_{k+1}} = \sum_{i=0}^{2n} W_i^{(c)} (\chi_{k+1|k}^{(i)} - \bar{w}_{k+1|k})(\mathcal{J}_{k+1|k}^{(i)} - \bar{y}_{k+1|k})^T \quad (11)$$

$$K_{k+1} = P_{x_{k+1} y_{k+1}} P_{y_{k+1} y_{k+1}}^{-1} \quad (12)$$

$$x_{k+1|k+1} = \bar{x}_{k+1|k} + K_{k+1} (y_{k+1} - \bar{y}_{k+1|k}) \quad (13)$$

$$P_{k+1|k+1} = P_{k+1|k} - K_{k+1} P_{y_{k+1} y_{k+1}}^{-1} K_{k+1}^T \quad (14)$$

e. $k = k + 1$, return to step b

III. NEURAL NETWORK AIDED MANEUVERING TARGET TRACKING ALGORITHM

We further propose a NN aided adaptive tracking algorithm for tracking target with high nonlinearities and modeling ambiguity. The structure of the algorithm is shown in Fig. 2. In [15], the authors developed an adaptive state estimation technique using NN and an EKF. The EKF is used to estimate the states by introducing a dynamic system model, at the same time, using the EKF to train the NN. The NN is used to calculate the nonlinearities, mismodeled dynamics, higher order modes, and other unknown facts of a system. It's absorbing that the algorithm trains online, together with the system state estimation. Because the NN can adaptively model the mismatched part of the system, and better performance of the estimation of the system state is expectable.

In view of the superiority of UKF to EKF, We attempt to improve the algorithm in [15] under the framework of UKF and introduce the new algorithm to maneuvering target tracking problem. The state transition equation and observation equation are given as

$$x_{k+1} = f(x_k) + n_k \quad (15)$$

$$y_k = h(x_k) + v_k \quad (16)$$

where x_k is the unknown state of the system and y_k is the only observed information. w_k is the *process* noise and v_k is

observation noise, that are independent identical distributive. $f(\cdot)$ and $h(\cdot)$ are the true system dynamic models.

We denote e_k as the error between true state evolution model and the a priori known mathematical model $\hat{f}(x_k)$, namely, $e_k = f(x_k) - \hat{f}(x_k)$. The NN is used to approximate the error. We cannot measure e_k and x_k directly; however, we can adjust the weights of the NN through the observation if and only if the observability condition is met. When

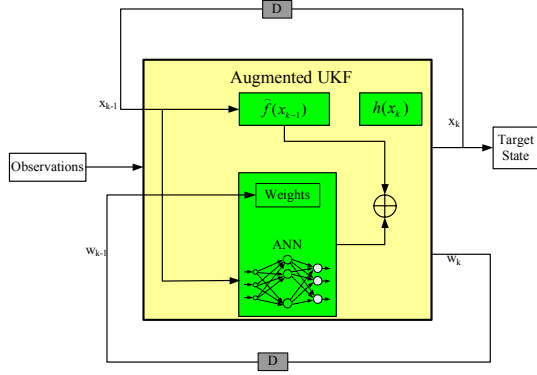


Fig. 2. The architecture of the adaptive maneuvering tracking algorithm. The module marked “D” denotes one-step time delay.

$e_k - f(x_k) \rightarrow 0$, the error e_k is well approximated, and the more accurate model becomes the sum of $\hat{f}(x_k)$ and the NN output. So we have

$$x_{k+1} = \hat{f}(x_k) + g(x_k, w_k) \quad (17)$$

$$w_{k+1} = w_k \quad (18)$$

If we represent the augmented state vector as $x_k^a = [x_k^T w_k^T]^T$, then the state transition equation can be redefined as

$$x_{k+1}^a = [x_{k+1}^T w_{k+1}^T]^T = \begin{bmatrix} \hat{f}(x_k) + g(x_k, w_k) \\ w_k \end{bmatrix} \triangleq f^a(x_k, w_k) \quad (19)$$

Because the observation is only related with the state x_k , (16) remains unchanged. Now the problem becomes obtaining the estimation of $\hat{x}_{k+1}^a$ based on the new state evolution equation (19) and the noisy observation (16). Once given $\hat{x}_0^a = E[x_0^a]$ and $P_0^a = E[(x_0^a - \hat{x}_0^a)(x_0^a - \hat{x}_0^a)^T]$, we can get $2L^a + 1$ sigma points (L^a is the dimension of x_k^a), then the filtering procedure of the NN aided adaptive tracking algorithm can be carried out recursively in the similar way as described in Section II.

IV. SIMULATION RESULTS

We have designed a simulating software platform for ground target tracking applications in wireless sensor network using MATLAB 7.0. The target moves through a field where many sensors have been deployed. For the sake of comparison, the target is tracked by both generic IMM

algorithm described in [2] and the NN-aided adaptive UKF, and then We will evaluate the tracking performance and the time complexity of different algorithms. In the IMM algorithm, a nearly constant velocity state evolution model and two constant turn models are used, and the filter kernel of each model is a normal UKF. Fig. 3 shows an example of the

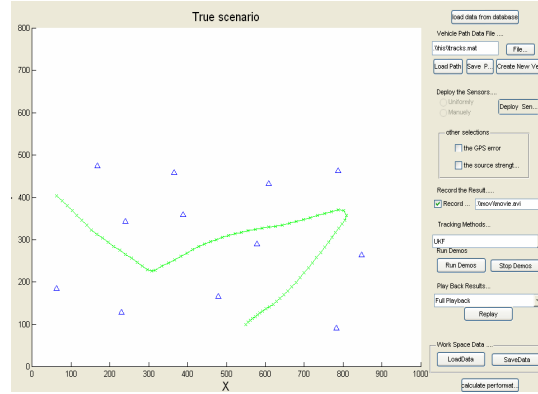


Fig. 3. The MATLAB simulating platform for target tracking in sensor networks. The unit of x-axis and y-axis is meter

simulating platform when a maneuvering target is moving through the sensor network field and the sensors are deployed manually.

We use the IMM algorithm and the NN-aided UKF algorithm to track the maneuvering target in the scenario shown in Fig. 3. For the latter, the neural network architecture is chosen by lots of experiments, which converges fast and has small computing burden to obtain a satisfying result. It is found that a NN with a 30-neuron structure in the hidden layer is suitable. The inputs of the NN are the current observations of chosen sensors. The target's state vector is defined as $x_k = (\xi_k, \eta_k, \dot{\xi}_k, \dot{\eta}_k)^T$, denoting the positions and velocities in

x and y axes. The initial state is $(550, 100, 5, 5)^T$. The targets moving time step length is 80s. The target makes a double turn maneuver, the turn rate is $45^\circ / s$ in the first turn and $22.5^\circ / s$ in the second turn. In other time the target moves with approximatively constant velocities. The turn maneuver is used to simulate the landform factor.

The measurements of the each sensor are the orientation and range information between the sensor and the target. Assuming the sampling period is $T_s = 1s$. The measurement at the i^{th} sensor at time step k can be expressed as

$$y_i(k) = \begin{bmatrix} \|r(k) - r_i\| \\ \arctan\left(\frac{\eta_k - \eta_i^s}{\xi_k - \xi_i^s}\right) \end{bmatrix} + v_i(k) \quad (20)$$

where $r(k)$ is a vector denoting the coordinates of the target at the time step k , namely, $(\xi_k, \eta_k)^T$. r_i is a vector denoting the coordinates of the i^{th} sensor, namely, $(\xi_i^s, \eta_i^s)^T$. $v_i(k)$ is the observation noise of $y_i(k)$, and we assume that each

sensor measurement noise is independent on others.

At each time step, the sensors having detected the target will transmit measurements to a lead node or processing center. Then, the sensors which are most closely to the target are chosen to form the tracker (they have the highest signal to noise ratio). Over time, the sensors which consist of the tracker change along with the target motion. To show the robust performance of our algorithm, we assume the target acceleration variance and sensor observation noise variance are not accurate modeled for both algorithms. Fig. 4 shows the true target trajectory together with the estimates of two algorithms. Intuitively, we can see that the proposed algorithm has smaller position errors than the IMM algorithm,

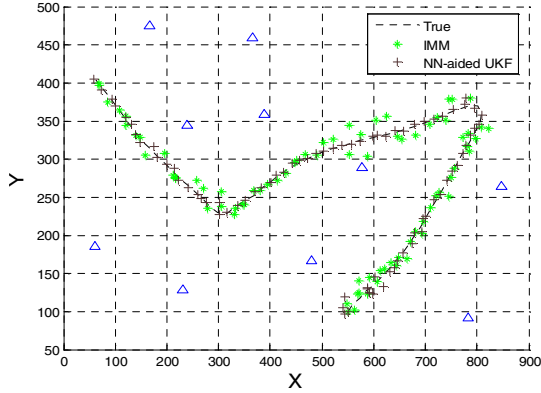


Fig. 4. The simulation results of trajectory for one run. The up triangles denote the positions of deployed sensors.

especially when the target is maneuvering.

Using $N_{MC}=100$ Monte Carlo simulations for each filter, the simulation results are given below in form of the position root mean square errors (RMSE) for the NN-aided adaptive UKF and the UKF-based IMM algorithms. The RMSE is defined as follows, and the value of each algorithm is shown in Table. I. The unit of the quantities is meter.

$$RMSE = \sqrt{\frac{1}{L} \sum_{k=1}^L \frac{1}{N_{MC}} \sum_{j=1}^{N_{MC}} ((\hat{\xi}_k^j - \xi_k^{true})^2 + (\hat{\eta}_k^j - \eta_k^{true})^2)} \quad (21)$$

where $L=80$ is the total simulation time steps. $\hat{\xi}_k^j, \hat{\eta}_k^j$ are

TABLE I RMSE FOR 100 MONTE CARLO SIMULATIONS		
Algorithm	IMM	NN aided UKF
RMSE	15.8026	7.3578

the filter position estimations at time step k in Monte Carlo run i . $\xi_k^{true}, \eta_k^{true}$ are the true target position at time step k .

The position estimation RMSE values of each filter presented for each time step is also given in Fig.5, according to the following equation,

$$RMSE(t) = \sqrt{\frac{1}{N_{MC}} \sum_{j=1}^{N_{MC}} ((\hat{\xi}_k^j - \xi_k^{true})^2 + (\hat{\eta}_k^j - \eta_k^{true})^2)} \quad (22)$$

From the above simulation results, we can see that: At the

beginning, the NN is laid in the convergence step and there are larger estimate errors than the IMM algorithm. But when the NN has converged, the proposed algorithm achieves better performance than the IMM algorithm even the target is maneuvering. We also find that the NN-aided adaptive UKF algorithm is robust against the model parameters errors, which shows that the proposed algorithm is very effective for target tracking problems with ambiguities and highly

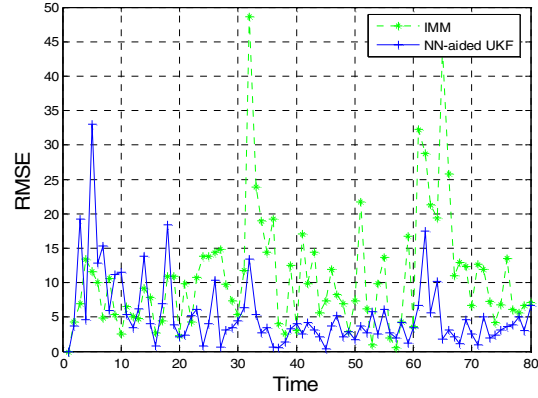


Fig. 5. The results of the IMM and NN-aided UKF algorithms for tracking maneuvering target.

maneuverability.

Besides the estimation performance, we also consider the computation complexity of our method. The computation complexity Ω is defined as the average execution time per time step in each random run, namely,

$$\Omega = \frac{1}{N_{MC}L} \sum_{j=1}^{N_{MC}} T_{sim,i} \quad (23)$$

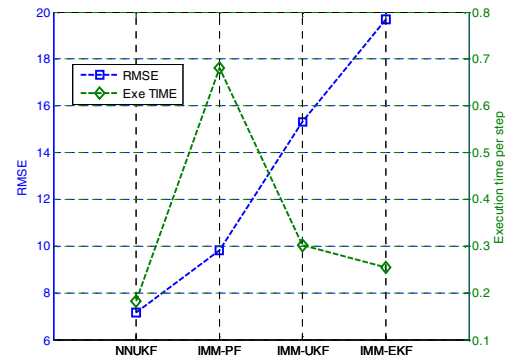


Fig. 6. The computation complexity vs. the position RMSE values for four different algorithm: NN-aided UKF, IMM-PF, IMM-UKF and IMM-EKF

where $T_{sim,i}$ is the total simulation time for the i^{th} Monte Carlo run.

Fig. 6 shows the computation complexity as well as the RMSE values of our NN-aided adaptive UKF and three different configured IMM algorithms. In the three IMM algorithms, the filter components for each state evolution model are EKF, UKF and generic particle filter (PF, see [16] for details), respectively. For the sake of notational

convenience, they are denoted as IMM-EKF, IMM-UKF and IMM-PF. The number of particles is 600 and residual resampling scheme is used in IMM-PF. As shown in Fig. 6, the average time consumption of proposed NN-aided UKF algorithm is below 0.18s for each time step, while the IMM-PF's execution time per step is about 0.68s, moreover, the RMSE of our method is also smaller than other algorithms. The results mean that the proposed algorithm can reach better estimation accuracy with less computation time, making the new method more suitable for real-time target tracking applications.

V. CONCLUSION

In this paper, we propose a new neural network aided adaptive unscented Kalman filter for target tracking with ambiguities and maneuverability. In this approach, a more robust learning algorithm is derived based on the UKF algorithm for NN training, which can give a more accurate estimate of the connecting weights of the NN, and the UKF is used for both the NN online training and target state filtering simultaneously. The NN is used to approximate the uncertainty of the system model due to mismodeling, extreme nonlinearities, and so on. Simulation results have show that the proposed algorithm is a very powerful tool to tackle maneuvering target tracking problems.

REFERENCES

- [1] Y. Bar-Shalom and X.-R. Li, *Estimation and Tracking: Principles, Techniques, and Software*, Artech House, 1993
- [2] E. Mazar, A. Averbuch, Y. B.-K., and J. Dayan, "Interacting multiple model methods in target tracking: a survey," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 34, no. 1, pp. 103-123, 1998
- [3] D. Li, K. D. Wong, Y. H. Hu, and A. M. Sayeed, "Detection, Classification, and Tracking of Targets in Distributed Sensor Networks," *IEEE Signal Processing Magazine*, vol. 19, no. 2, pp. 17-29, 2002
- [4] B.D. Anderson and J.B. Moore, *Optimal Filtering*, Prentice-Hall, New Jersey, 1979
- [5] H.W. Sorenson, *Kalman filtering: Theory and application*. IEEE Press, New York, 1985
- [6] D. Lerro, Y. Bar-Shalom, "Tracking with Debaised Consistent Converted Measurement vs EKF," *IEEE Trans on Aerospace and Electronics Systems*, vol. 29, no. 3, pp. 1015-1022, 1993
- [7] S.J. Julier, J.K. Uhlmann, and H.F. Durrant-Whyten, "A new approach for filtering nonlinear system," *Proc of the American Control Conf.*, pp. 1628-1632, 1995
- [8] S. J. Julier and J.K. Uhlmann, "A New Extension of The Kalman Filter to Nonlinear Systems," *The 11th Int Symposium on Aerospace/Defense Sensing, Simulation and Controls*, pp. 54-65, Orlando, 1997
- [9] T. Lefebvre, H. Bruyninckx, and J. De Schutter, "Comment on "A New Method for The Nonlinear Transformation of Means and Covariances in Filters and Estimators"," *IEEE Trans on Automatic Control*, vol. 47, no. 8, pp. 1406-1408, 2002
- [10] S.J. Julier, J.K. Uhlmann, "The Scaled Unscented Transformation," *Proc. of American Control Conf.*, pp. 4555-4559, Jefferson City, 2002
- [11] P. Kuczzeski, "Neural Network Approaches to Multitarget Tracking," *Proc. of the IEEE ICNN Conference*, 1987
- [12] Leonard Chin, "Application of Neural Networks in Target Tracking Data Fusion," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 30, no. 1, pp. 281-287, 1994
- [13] Y. Iiguni, H. Sakai, H. Tokumaru, "A real-time learning algorithm for a multilayered neural network based on the extended Kalman filter," *IEEE Trans. Signal Process.*, vol. 40, no. 4, pp. 959-966, 1992
- [14] H. Wabgaonkar, and A. Stubberud, "Approximation and estimation techniques for neural networks," *Proc. 29th IEEE Conf. on Decision and Control*, pp. 1628-1632, 1995
- [15] R. N. Lobbia, S. C. Stubberud, M. W. Owen, "Adaptive extended Kalman filter using artificial neural networks", *Int. J. Smart Eng. Syst. Des.*, vol. 1, pp. 207-221, 1998
- [16] M. S. Arulampalam, S. Maskell, N. Gordon, etc, "A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking", *IEEE Trans. on Signal Processing*, vol. 50, no. 2, pp. 174-188, 1998